

從 App Engine 工作佇列提取工作遷移至 Cloud Pub/Sub (模組 19)

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-19-pubsub?hl=zh-tw>

步驟數：8

瞭解如何遷移 Python 2 App Engine NDB & 從工作佇列 (提取工作) 應用程式傳送至 Cloud NDB (&A)；再升級至 Python 3

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 建立 Pub/Sub 資源](#)
5. [5. 更新設定](#)
6. [6. 修改應用程式程式碼](#)
7. [7. 摘要/清除](#)
8. [8. 其他資源](#)

1. 總覽

[無伺服器遷移工作站系列程式碼研究室](#) (自學式實作教學課程) 和[相關影片](#)，旨在協助 [Google Cloud 無伺服器](#)開發人員完成一或多項遷移作業 (主要是從舊版服務遷移)，進而翻新應用程式。這樣做可提高應用程式的可攜性，並提供更多選項和彈性，讓您整合及存取更多 Cloud 產品，並更輕鬆地升級至新版語言。雖然一開始的重點是早期 Cloud 使用者，主要是 [App Engine](#) (標準環境) 開發人員，但本系列涵蓋範圍廣泛，也包括其他無伺服器平台，例如 [Cloud Functions](#) 和 [Cloud Run](#)，或適用於其他平台。

本程式碼研究室旨在向 Python 2 App Engine 開發人員說明，如何從 App Engine 工作佇列提取工作遷移至 [Cloud Pub/Sub](#)。此外，從 App Engine NDB 遷移至 [Cloud NDB](#) 以存取 Datastore (主要涵蓋在第 2 模組中) 時，也會[隱含遷移](#)，並升級至 Python 3。

在第 18 個單元中，您瞭解如何在應用程式中新增 **提取**工作的使用方式。在本單元中，您將採用完成的第 18 個單元應用程式，並將該使用方式遷移至 Cloud Pub/Sub。如果使用工作佇列執行**推送**工作，請改用 Cloud Tasks，並參閱第 7 至 9 個單元。

未使用工作佇列？

如果您的應用程式未使用 App Engine 工作佇列服務，請略過第 7 到 9 節 (推送工作) 和第 18 到 19 節 (提取工作)，或將這些程式碼研究室當做練習，熟悉工作佇列遷移作業。

在接下來的研究室中

- 將 [App Engine 工作佇列 \(提取工作\)](#) 的使用方式替換為 [Cloud Pub/Sub](#)
- 將 [App Engine NDB](#) 的使用方式替換為 [Cloud NDB](#) (另請參閱第 2 節)
- 將應用程式移植到 Python 3

軟硬體需求

- 擁有 [Google Cloud Platform 專案](#)，且已啟用 [GCP 帳單帳戶](#)
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發及部署](#) App Engine 應用程式的基本知識
- 可正常運作的第 18 堂課 App Engine 範例應用程式

問卷調查

2. 背景

[App Engine 工作佇列](#)支援推送和提取工作。為提升應用程式可攜性，Google Cloud 建議您從 Task Queue 等舊版套裝組合服務，遷移至其他 Cloud 獨立或第三方同等服務。

- [工作佇列發送工作](#)使用者應遷移至 [Cloud Tasks](#)。
- [工作佇列提取工作](#)的使用者應遷移至 [Cloud Pub/Sub](#)。

第 7 到 9 節的遷移模組涵蓋推送工作遷移，第 18 到 19 節則著重於提取工作遷移。雖然 Cloud Tasks 與工作佇列發送工作更為相近，但 Pub/Sub 與工作佇列提取工作並非如此。

Pub/Sub 的功能比工作佇列提供的提取功能更多。舉例來說，Pub/Sub 也具備*推送*功能，但 Cloud Tasks 更像是 Task Queue 推送工作，因此任何遷移模組都不會涵蓋 Pub/Sub 推送。本第 19 堂實驗室課程會示範如何將佇列機制從工作佇列提取佇列切換至 Pub/Sub，以及如何從 App Engine NDB 遷移至 Cloud NDB，以存取 Datastore，重複[第 2 堂](#)實驗室課程的遷移作業。

雖然第 18 堂課的程式碼「宣稱」是 Python 2 範例應用程式，但原始碼本身與 Python 2 和 3 相容，即使在第 19 堂課中遷移至 Cloud Pub/Sub (和 Cloud NDB) 後，情況依然如此。

本教學課程包含下列步驟：

1. 設定/準備工作
 2. 更新設定
 3. 修改應用程式程式碼
-

3. 設定/準備工作

本節將說明如何：

1. 設定 Cloud 專案
2. 取得基準範例應用程式
3. (重新) 部署及驗證基準應用程式
4. 啟用新的 Google Cloud 服務/API

這些步驟可確保您從可運作的程式碼開始，並準備好遷移至雲端服務。

1. 設定專案

如果您已完成[第 18 堂程式碼研究室](#)，請重複使用該專案 (和程式碼)。或者，您也可以[建立全新專案](#)，或重複使用其他現有專案。確認專案具備可用的帳單帳戶和已啟用的 App Engine 應用程式。找出專案 ID，因為在本程式碼研究室中，每當遇到 `PROJECT_ID` 變數時，您都需要使用該 ID。

2. 取得基準範例應用程式

其中一項必要條件是可正常運作的第 18 堂課 App Engine 應用程式，因此請完成該堂課的程式碼研究室 (建議您這麼做；連結如上)，或從存放區複製[第 18 堂課的程式碼](#)。無論使用你的或我們的，我們都會從這裡開始 (「START」)。本程式碼研究室會逐步說明遷移程序，最後提供類似於第 19 堂課存放區資料夾 ("FINISH") 內容的程式碼。

- 開始：[Module 18 資料夾](#) (Python 2)
- 完成：[第 19 堂課資料夾](#) (Python 2 和 3)
- [整個存放區](#) (複製或下載 [ZIP 檔案](#))

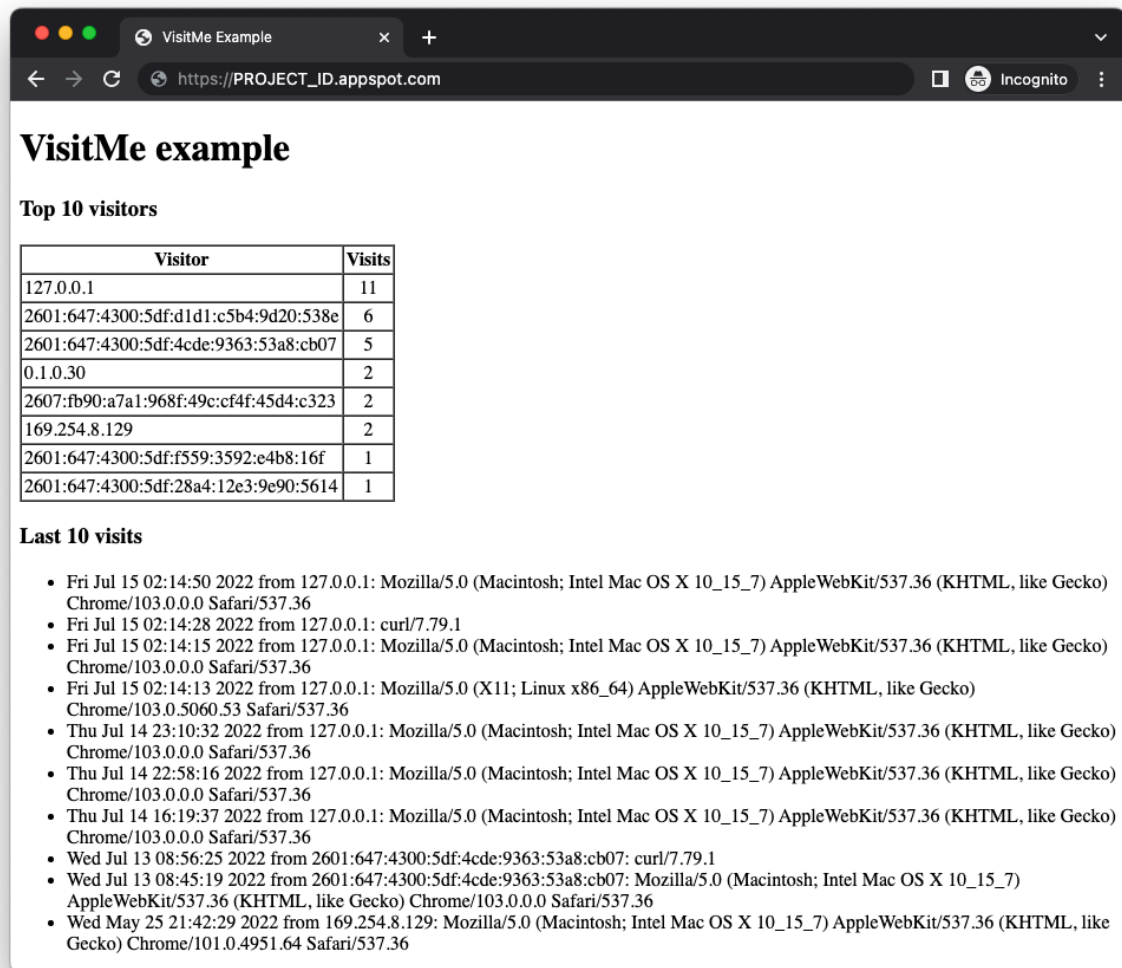
無論使用哪個 Module 18 應用程式，資料夾都應如下所示，可能也會有 `lib` 資料夾：

```
$ ls
README.md          appengine_config.py  queue.yaml          templates
app.yaml           main.py              requirements.txt
```

3. (重新) 部署及驗證基準應用程式

請按照下列步驟部署 Module 18 應用程式：

1. 刪除 `lib` 資料夾 (如有)，然後執行 `pip install -t lib -r requirements.txt` 重新填入 `lib`。如果開發電腦同時安裝 Python 2 和 3，可能需要改用 `pip2`。
2. 請確認您已[安裝及初始化](#) `gcloud` 指令列工具，並查看[其用法](#)。
3. (選用) 使用 `gcloud config set project PROJECT_ID` 設定 Cloud 專案，這樣您就不必在發出的每個 `gcloud` 指令中輸入 `PROJECT_ID`。
4. 使用 `gcloud app deploy` 部署範例應用程式
5. 確認應用程式是否正常運作。如果您已完成第 18 堂程式碼研究室，應用程式會顯示頂端訪客和最近的造訪記錄 (如下圖所示)。否則可能不會顯示任何訪客人數。



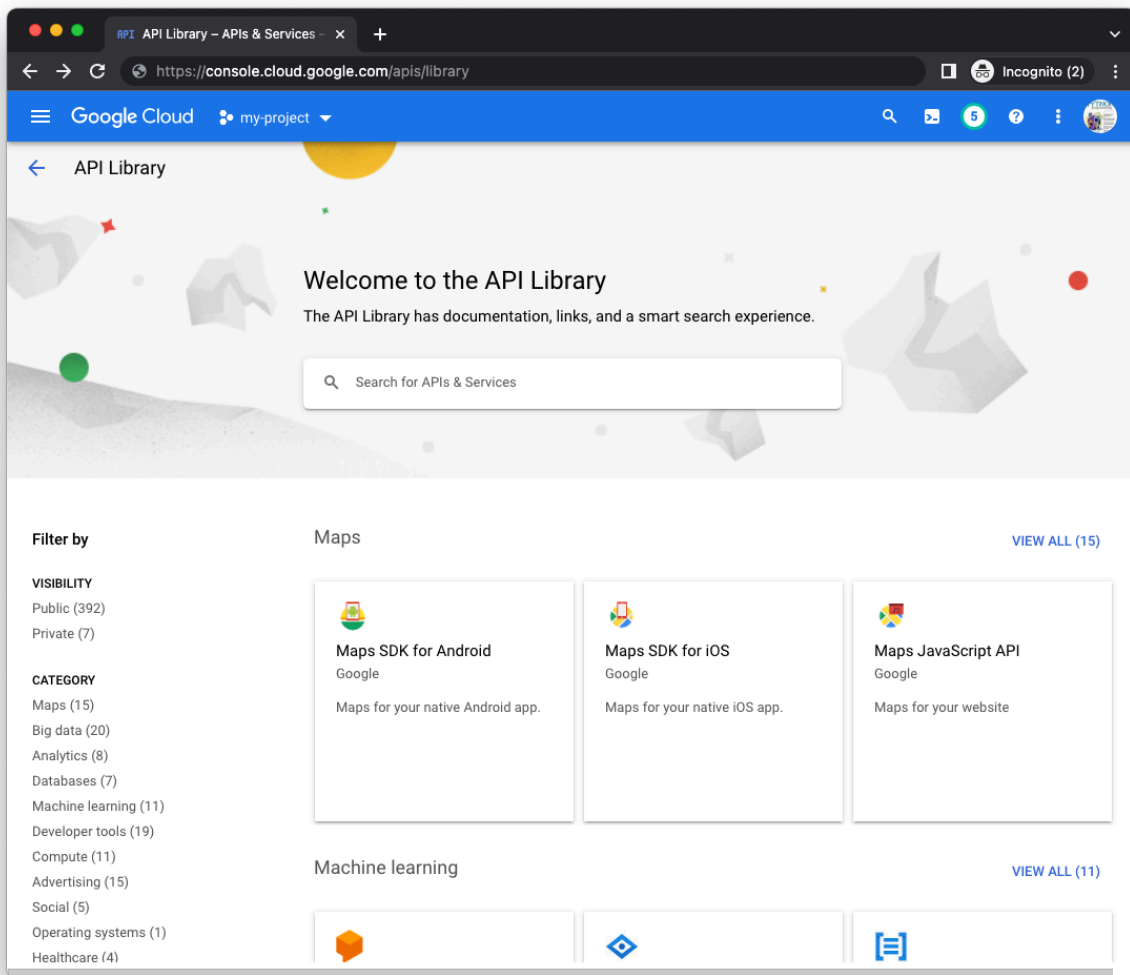
遷移第 18 課範例應用程式前，請先啟用修改後的應用程式會使用的 Cloud 服務。

4. 啟用新的 Google Cloud 服務/API

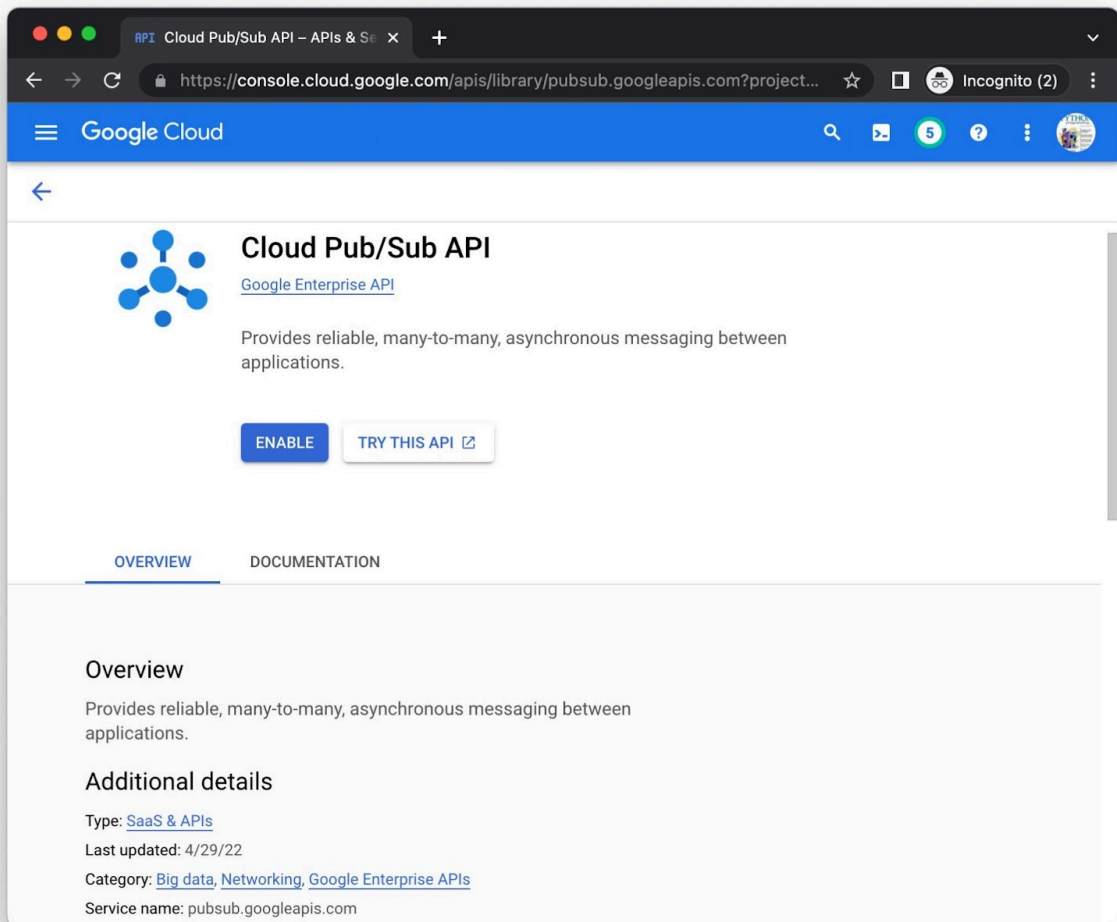
舊版應用程式使用 App Engine 隨附服務，不需要額外設定，但獨立的 Cloud 服務則需要設定。更新後的應用程式會同時使用 Cloud Pub/Sub 和 [Cloud Datastore](#) (透過 Cloud NDB 用戶端程式庫)。App Engine 和這兩個 Cloud API 都有「永久免費」方案配額，只要不超過這些限制，完成本教學課程就不會產生費用。您可以透過 Cloud 控制台或指令列啟用 Cloud API，視個人偏好而定。

透過 Cloud 控制台

在 Cloud 控制台中，前往 [API 管理工具](#) 的「程式庫」頁面 (適用於正確的專案)，然後使用頁面中間的搜尋列，搜尋 Cloud Datastore 和 Cloud Pub/Sub API：



分別點選各個 API 的「啟用」按鈕，系統可能會提示您提供帳單資訊。舉例來說，這是 Cloud Pub/Sub API 程式庫頁面：



使用指令列

雖然透過控制台啟用 API 具有視覺上的資訊性，但有些人偏好使用指令列。發出 `gcloud services enable pubsub.googleapis.com datastore.googleapis.com` 指令，同時啟用這兩個 API：

```
$ gcloud services enable pubsub.googleapis.com datastore.googleapis.com
Operation "operations/acat.p2-aaa-bbb-ccc-ddd-eee-ffffff" finished successfully.
```

系統可能會提示您輸入帳單資訊。如要啟用其他 Cloud API 並瞭解其 URI，請前往各 API 的程式庫頁面底部。舉例來說，請觀察上方 Pub/Sub 頁面底部的「服務名稱」`pubsub.googleapis.com`。

完成這些步驟後，專案就能存取 API。現在可以更新應用程式，**使用這些 API** 了。

4. 建立 Pub/Sub 資源

回顧第 18 堂課的 Task Queue 工作流程順序：

1. 第 18 個單元使用 `queue.yaml` 檔案建立名為 `pullq` 的提取佇列。
2. 應用程式會將工作新增至提取佇列，以便追蹤訪客。
3. 工作最終會由工作站處理，租用時間有限 (一小時)。
4. 系統會執行工作，計算近期訪客人數。
5. 工作完成後，系統會從佇列中刪除。

您將使用 Pub/Sub 複製類似的工作流程。下一節將介紹基本的 Pub/Sub 術語，並提供三種建立必要 Pub/Sub 資源的方式。

App Engine 工作佇列 (提取) 與 Cloud Pub/Sub 術語

改用 Pub/Sub 時，詞彙需要稍做調整。下方列出主要類別，以及這兩項產品的相關字詞。此外，也請參閱[遷移指南](#)，其中也提供類似的比較。

- **資料排隊結構**：使用工作佇列時，資料會進入**提取佇列**；使用 Pub/Sub 時，資料會進入**主題**。
- **佇列資料單位**：工作佇列中的**提取工作**在 Pub/Sub 中稱為**訊息**。
- **資料處理器**：使用工作佇列時，**工作人員**會存取提取工作；使用 Pub/Sub 時，您需要**訂閱項目/訂閱者**才能接收訊息。
- **資料擷取**：**租用**提取工作與從主題**提取**訊息 (透過訂閱) 相同。
- **清除/完成**：完成後**刪除**提取佇列中的工作佇列工作，等同於**確認** Pub/Sub 訊息。

雖然排隊產品有所變更，但工作流程大致相同：

1. 應用程式使用的是名為 `pullq` 的**主題**，而非提取佇列。
2. 應用程式會將**訊息**傳送至主題 (`pullq`)，而不是將工作新增至提取佇列。
3. 與其讓工作站從提取佇列租用工作，不如讓名為 `worker` 的**訂閱者**從 `pullq` 主題**提取**訊息。
4. 應用程式會處理訊息酬載，並在 Datastore 中遞增訪客人數。
5. 應用程式會**確認**已處理的訊息，而不是從提取佇列中刪除工作。

使用工作佇列時，設定程序包括建立提取佇列。使用 Pub/Sub 時，設定需要建立主題和訂閱項目。在第 18 堂課中，我們在應用程式執行作業以外處理了 `queue.yaml`，現在必須對 Pub/Sub 執行相同操作。

您可以透過下列三種方式建立主題和訂閱項目：

1. 透過 Cloud 控制台
2. 使用指令列，或
3. 從程式碼 (簡短的 Python 指令碼)

請選擇下列其中一個選項，然後按照相應的操作說明建立 Pub/Sub 資源。

透過 Cloud 控制台

如要透過 Cloud 控制台建立主題，請按照下列步驟操作：

1. 前往 Cloud 控制台的 [Pub/Sub 主題頁面](#)。
2. 按一下頂端的「建立主題」，開啟新的對話方塊 (如下圖所示)
3. 在「Topic ID」(主題 ID) 欄位中輸入 `pullq`。

- 取消選取所有已勾選的選項，然後選取「Google-managed encryption key」(Google 代管的加密金鑰)。
- 按一下「建立主題」按鈕。

主題建立對話方塊如下所示：

Create a topic

A topic forwards messages from publishers to subscribers.

Topic ID * ?

! ID is required

- ☐ Add a default subscription ?
- ☐ Use a schema ?
- ☐ Set message retention duration (not free) ?

Encryption

- ☒ Google-managed encryption key
No configuration required
- ☐ Customer-managed encryption key (CMEK)
Manage via Google Cloud Key Management Service

CANCEL

CREATE TOPIC

現在您已建立主題，接下來必須為該主題建立訂閱項目：

- 前往 Cloud 控制台的 [Pub/Sub 訂閱項目頁面](#)。
- 按一下頂端的「建立訂閱項目」(請參閱下圖)。
- 在「Subscription ID」(訂閱項目 ID) 欄位中輸入 `worker`。
- 從「Select a Cloud Pub/Sub topic」(選取 Cloud Pub/Sub 主題) 下拉式選單中選擇 `pullq`，並記下其「完整路徑名稱」，例如 `projects/PROJECT_ID/topics/pullq`
- 在「傳送類型」中，選取「提取」。
- 其餘選項維持原狀，然後按一下「建立」按鈕。

訂閱方案建立畫面如下所示：

Google Cloud my-project

Create subscription

A subscription directs messages on a topic to subscribers. Messages can be pushed to subscribers immediately, or subscribers can pull messages as needed.

Subscription ID *

worker

?

Subscription name: projects/[redacted]/subscriptions/worker

Select a Cloud Pub/Sub topic *

projects/[redacted]/topics/pullq

▼

Delivery type ?

☒ Pull

☐ Push

Message retention duration ?

Duration is from

Days

7

Minutes

0

Retry policy

Retry policy will be used for events that exceed the configured deadline exceeded events

☒ Retry immediately

☐ Retry after exponential backoff delay

CREATE

Additional Pub/Sub subscription options (take all defaults)

您也可以從「主題」頁面建立訂閱項目，這個「捷徑」或許有助於您將主題與訂閱項目建立關聯。如要进一步瞭解如何建立訂閱項目，請參閱[說明文件](#)。

使用指令列

Pub/Sub 使用者可以分別使用 `gcloud pubsub topics create TOPIC_ID` 和 `gcloud pubsub subscriptions create SUBSCRIPTION_ID --topic= TOPIC_ID` 指令建立主題和訂閱項目。使用 `pullq` 的 `TOPIC_ID` 和 `worker` 的 `SUBSCRIPTION_ID` 執行這些作業，會產生專案 `PROJECT_ID` 的下列輸出內容：

```
$ gcloud pubsub topics create pullq
Created topic [projects/PROJECT_ID/topics/pullq].

$ gcloud pubsub subscriptions create worker --topic=pullq
Created subscription [projects/PROJECT_ID/subscriptions/worker].
```

另請參閱快速入門導覽課程說明文件中的[這個頁面](#)。如果定期建立主題和訂閱項目，使用指令列可能會簡化工作流程，而且這類指令可用於 Shell 指令碼。

從程式碼 (簡短的 Python 指令碼)

若要自動建立主題和訂閱項目，也可以在原始碼中使用 Pub/Sub API。以下是 Module 19 存放區資料夾中的 [maker.py](#) 指令碼程式碼。

```
from __future__ import print_function
import google.auth
from google.api_core import exceptions
from google.cloud import pubsub

_, PROJECT_ID = google.auth.default()
TOPIC = 'pullq'
SBSCR = 'worker'

ppc_client = pubsub.PublisherClient()
psc_client = pubsub.SubscriberClient()
TOP_PATH = ppc_client.topic_path(PROJECT_ID, TOPIC)
SUB_PATH = psc_client.subscription_path(PROJECT_ID, SBSCR)

def make_top():
    try:
        top = ppc_client.create_topic(name=TOP_PATH)
        print('Created topic %r (%s)' % (TOPIC, top.name))
    except exceptions.AlreadyExists:
        print('Topic %r already exists at %r' % (TOPIC, TOP_PATH))

def make_sub():
    try:
        sub = psc_client.create_subscription(name=SUB_PATH, topic=TOP_PATH)
        print('Subscription created %r (%s)' % (SBSCR, sub.name))
    except exceptions.AlreadyExists:
        print('Subscription %r already exists at %r' % (SBSCR, SUB_PATH))
    try:
        psc_client.close()
    except AttributeError: # special Py2 handler for grpcio<1.12.0
        pass

make_top()
make_sub()
```

執行這個指令碼會產生預期輸出內容 (前提是沒有錯誤)：

```
$ python3 maker.py
Created topic 'pullq' (projects/PROJECT_ID/topics/pullq)
Subscription created 'worker' (projects/PROJECT_ID/subscriptions/worker)
```

呼叫 API 建立已存在的資源時，用戶端程式庫會擲回 `google.api_core.exceptions.AlreadyExists` 例外狀況，而指令碼會妥善處理：

```
$ python3 maker.py
Topic 'pullq' already exists at 'projects/PROJECT_ID/topics/pullq'
Subscription 'worker' already exists at 'projects/PROJECT_ID/subscriptions/worker'
```

如果您是 Pub/Sub 新手，請參閱 [Pub/Sub 架構白皮書](#)，進一步瞭解相關資訊。

步驟 5 · 約 10 分鐘

5. 更新設定

設定更新包括變更各種設定檔，以及在 Cloud Pub/Sub 生態系統中建立等同於 App Engine 提取佇列的項目。

刪除 queue.yaml

我們將完全停用工作佇列，因此請刪除 `queue.yaml`，因為 Pub/Sub 不會使用這個檔案。您要建立 [Pub/Sub 主題](#) (和[訂閱](#))，而不是建立[提取佇列](#)。

requirements.txt

將 `google-cloud-ndb` 和 `google-cloud-pubsub` 都附加至 `requirements.txt`，以便從模組 18 加入 `flask`。更新後的第 19 堂課 `requirements.txt` 應如下所示：

```
flask
google-cloud-ndb
google-cloud-pubsub
```

這個 `requirements.txt` 檔案沒有任何版本號碼，表示系統會選取最新版本。如果發生任何不相容的情況，請按照標準做法，使用版本號碼鎖定應用程式的可用版本。

app.yaml

`app.yaml` 的變更內容會因您繼續使用 Python 2 或升級至 Python 3 而有所不同。

Python 2

上述 `requirements.txt` 更新會新增 Google Cloud 用戶端程式庫的使用方式。這些功能需要 App Engine 的額外支援，也就是幾個[內建程式庫](#)、`setuptools` 和 `grpcio`。如要使用內建程式庫，請在 `app.yaml` 中加入 `libraries` 區段和程式庫版本號碼，或使用「latest」表示 App Engine 伺服器上可用的最新版本。模組 18 `app.yaml` 還沒有這些區段：

BEFORE:

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app
```

在 `app.yaml` 中新增 `libraries` 區段，並為 `setuptools` 和 `grpcio` 新增項目，選取最新版本。此外，請為 Python 3 新增預留位置 `runtime` 項目，並連同目前的 3.x 版本 (例如撰寫本文時的 3.10) 一併註解。完成這些變更後，`app.yaml` 現在看起來會像這樣：

修改後：

```
#runtime: python310
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app

libraries:
- name: setuptools
  version: latest
- name: grpcio
  version: latest
```

Python 3

如果是 Python 3 使用者和 `app.yaml`，則一切都與移除項目有關。在本節中，您將刪除 `handlers` 區段、`threadsafe` 和 `api_version` 指令，且不會建立 `libraries` 區段。

第二代執行階段[不提供內建的第三方程式庫](#)，因此 `app.yaml` 中**不需要** `libraries` 區段。此外，您也不再需要複製(有時稱為供應商或自行組合)[非內建的第三方套件](#)。您只需要在 `requirements.txt` 中列出應用程式使用的第三方程式庫。

`app.yaml` 中的 `handlers` 區段用於指定應用程式(指令碼)和靜態檔案處理常式。由於 Python 3 執行階段需要網頁架構自行執行轉送作業，因此所有指令碼處理常式都必須變更為 `auto`。如果您的應用程式(例如第 18 集的應用程式)不提供靜態檔案，所有路徑都會是 `auto`，因此不相關。因此也不需要 `handlers` 區段，請將其刪除。

最後，Python 3 不會使用 `threadsafe` 和 `api_version` 指令，因此請一併刪除。總而言之，您應該刪除 `app.yaml` 的所有區段，只留下 `runtime` 指令，並指定新版 Python 3，例如 3.10。以下是更新前後的 `app.yaml` 外觀：

BEFORE:

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app
```

修改後：

```
runtime: python310
```

如果您還不想從 Python 3 的 `app.yaml` 中刪除所有內容，我們在「Module 19」存放區資料夾中提供了 `app3.yaml` 替代檔案。如要改用這個檔案進行部署，請務必在指令結尾附加這個檔案名稱：`gcloud app deploy app3.yaml` (否則系統會預設使用您未變更的 Python 2 `app.yaml` 檔案部署應用程式)。

appengine_config.py

如果您要升級至 Python 3，就不需要 `appengine_config.py`，請將其刪除。不必這麼做的原因是，第三方程式庫支援功能只需要在 `requirements.txt` 中指定這些程式庫。Python 2 使用者請繼續閱讀。

模組 18 `appengine_config.py` 包含[支援第三方程式庫的適當程式碼](#)，例如 Flask 和剛才新增至 `requirements.txt` 的 Cloud 用戶端程式庫：

BEFORE:

```
from google.appengine.ext import vendor

# Set PATH to your libraries folder.
PATH = 'lib'
# Add libraries installed in the PATH folder.
vendor.add(PATH)
```

不過，光是這個程式碼還不足以支援剛加入的內建程式庫 (`setuptools`、`grpcio`)。您還需要幾行程式碼，因此請更新 `appengine_config.py`，使其看起來像這樣：

修改後：

```
import pkg_resources
from google.appengine.ext import vendor

# Set PATH to your libraries folder.
PATH = 'lib'
# Add libraries installed in the PATH folder.
vendor.add(PATH)
# Add libraries to pkg_resources working set to find the distribution.
pkg_resources.working_set.add_entry(PATH)
```

若要進一步瞭解支援 Cloud 用戶端程式庫所需的變更，請參閱[遷移套裝服務說明文件](#)。

其他設定更新

如果有 `lib` 資料夾，請刪除。如果您是 Python 2 使用者，請發出下列指令，補充 `lib` 資料夾：

```
pip install -t lib -r requirements.txt # or pip2
```

如果開發系統同時安裝 Python 2 和 3，可能需要使用 `pip2`，而非 `pip`。

6. 修改應用程式程式碼

本節將介紹主要應用程式檔案 `main.py` 的更新，以 Cloud Pub/Sub 取代 App Engine 工作佇列提取佇列。網頁範本「`templates/index.html`」沒有任何變更。這兩個應用程式的運作方式應完全相同，顯示的資料也一樣。

更新匯入作業和初始化

匯入和初始化作業有幾項更新：

1. 在匯入項目中，將 App Engine NDB 和 Task Queue 換成 Cloud NDB 和 Pub/Sub。
2. 將 `pullq` 從 `QUEUE` 名稱重新命名為 `TOPIC` 名稱。
3. 使用提取工作時，工作站會租用工作一小時，但使用 Pub/Sub 時，逾時是依訊息計算，因此請刪除 `HOURL` 常數。
4. Cloud API 需要使用 API 用戶端，因此請為 Cloud NDB 和 Cloud Pub/Sub 啟動這些用戶端，後者會為主題和訂閱項目提供用戶端。
5. Pub/Sub 需要 Cloud 專案 ID，因此請從 `google.auth.default()` 匯入並取得該 ID。
6. Pub/Sub 需要主題和訂閱項目的「完整路徑名稱」，因此請使用 `*_path()` 便利函式建立這些項目。

以下是第 18 堂課的匯入和初始化作業，以及實作上述變更後各節的樣貌，其中大部分的新程式碼都是各種 Pub/Sub 資源：

BEFORE:

```
from flask import Flask, render_template, request
from google.appengine.api import taskqueue
from google.appengine.ext import ndb

HOURL = 3600
LIMIT = 10
TASKS = 1000
QNAME = 'pullq'
QUEUE = taskqueue.Queue(QNAME)
app = Flask(__name__)
```

修改後：

```

from flask import Flask, render_template, request
import google.auth
from google.cloud import ndb, pubsub

LIMIT = 10
TASKS = 1000
TOPIC = 'pullq'
SBSCR = 'worker'

app = Flask(__name__)
ds_client = ndb.Client()
ppc_client = pubsub.PublisherClient()
psc_client = pubsub.SubscriberClient()
_, PROJECT_ID = google.auth.default()
TOP_PATH = ppc_client.topic_path(PROJECT_ID, TOPIC)
SUB_PATH = psc_client.subscription_path(PROJECT_ID, SBSCR)

```

查看資料模型更新

`visit` 資料模型不會變更。如要存取 Datastore，必須明確使用 Cloud NDB API 用戶端內容管理工具 `ds_client.context()`。在程式碼中，這表示您會在 Python `with` 區塊內，將 Datastore 呼叫包裝在 `store_visit()` 和 `fetch_visits()` 中。這項更新與第 2 模組涵蓋的內容相同。

對 Pub/Sub 而言，最相關的變更就是將工作佇列提取工作排入佇列，改為將 Pub/Sub 訊息發布至 `pullq` 主題。以下是更新前後的程式碼：

BEFORE:

```

class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor = ndb.StringProperty()
    timestamp = ndb.DateTimeProperty(auto_now_add=True)

def store_visit(remote_addr, user_agent):
    'create new Visit in Datastore and queue request to bump visitor count'
    Visit(visitor='{}: {}'.format(remote_addr, user_agent)).put()
    QUEUE.add(taskqueue.Task(payload=remote_addr, method='PULL'))

def fetch_visits(limit):
    'get most recent visits'
    return Visit.query().order(-Visit.timestamp).fetch(limit)

```

修改後：

```

class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor = ndb.StringProperty()
    timestamp = ndb.DateTimeProperty(auto_now_add=True)

def store_visit(remote_addr, user_agent):
    'create new Visit in Datastore and queue request to bump visitor count'
    with ds_client.context():
        Visit(visitor='{ }: { }'.format(remote_addr, user_agent)).put()
    ppc_client.publish(TOP_PATH, remote_addr.encode('utf-8'))

def fetch_visits(limit):
    'get most recent visits'
    with ds_client.context():
        return Visit.query().order(-Visit.timestamp).fetch(limit)

```

VisitorCount 資料模型更新

VisitorCount 資料模型不會變更，但 Datastore 查詢會包裝在 `with` 區塊中，如下所示：`fetch_counts()`

BEFORE:

```

class VisitorCount(ndb.Model):
    visitor = ndb.StringProperty(repeated=False, required=True)
    counter = ndb.IntegerProperty()

def fetch_counts(limit):
    'get top visitors'
    return VisitorCount.query().order(-VisitorCount.counter).fetch(limit)

```

修改後：

```

class VisitorCount(ndb.Model):
    visitor = ndb.StringProperty(repeated=False, required=True)
    counter = ndb.IntegerProperty()

def fetch_counts(limit):
    'get top visitors'
    with ds_client.context():
        return VisitorCount.query().order(-VisitorCount.counter).fetch(limit)

```

更新工作人員代碼

工作人員程式碼會更新，將 NDB 替換為 Cloud NDB，並將工作佇列替換為 Pub/Sub，但工作流程維持不變。

1. 將 Datastore 呼叫包裝在 Cloud NDB 環境管理員 `with` 區塊中。
2. 工作佇列清理作業包括刪除提取佇列中的所有工作。使用 Pub/Sub 時，「確認 ID」會收集在 `acks` 中，然後在結尾刪除/確認。
3. 工作佇列提取工作的方式，與提取 Pub/Sub 訊息類似。刪除提取工作時，系統會一併刪除工作物件本身，但刪除 Pub/Sub 訊息時，系統會透過確認 ID 刪除訊息。

4. Pub/Sub 訊息酬載需要位元組 (而非 Python 字串)，因此發布至主題和從主題提取訊息時，分別需要進行一些 UTF-8 編碼和解碼作業。

使用下列更新後的程式碼取代 `log_visitors()`，實作剛才說明的變更：

BEFORE:

```
@app.route('/log')
def log_visitors():
    'worker processes recent visitor counts and updates them in Datastore'
    # tally recent visitor counts from queue then delete those tasks
    tallies = {}
    tasks = QUEUE.lease_tasks(HOUR, TASKS)
    for task in tasks:
        visitor = task.payload
        tallies[visitor] = tallies.get(visitor, 0) + 1
    if tasks:
        QUEUE.delete_tasks(tasks)

    # increment those counts in Datastore and return
    for visitor in tallies:
        counter = VisitorCount.query(VisitorCount.visitor == visitor).get()
        if not counter:
            counter = VisitorCount(visitor=visitor, counter=0)
            counter.put()
        counter.counter += tallies[visitor]
        counter.put()
    return 'DONE (with %d task[s] logging %d visitor[s])\r\n' % (
        len(tasks), len(tallies))
```

修改後：

```

@app.route('/log')
def log_visitors():
    'worker processes recent visitor counts and updates them in Datastore'
    # tally recent visitor counts from queue then delete those tasks
    tallies = {}
    acks = set()
    rsp = psc_client.pull(subscription=SUB_PATH, max_messages=TASKS)
    msgs = rsp.received_messages
    for rcvd_msg in msgs:
        acks.add(rcvd_msg.ack_id)
        visitor = rcvd_msg.message.data.decode('utf-8')
        tallies[visitor] = tallies.get(visitor, 0) + 1
    if acks:
        psc_client.acknowledge(subscription=SUB_PATH, ack_ids=acks)
    try:
        psc_client.close()
    except AttributeError: # special handler for grpcio<1.12.0
        pass

    # increment those counts in Datastore and return
    if tallies:
        with ds_client.context():
            for visitor in tallies:
                counter = VisitorCount.query(VisitorCount.visitor == visitor).get()
                if not counter:
                    counter = VisitorCount(visitor=visitor, counter=0)
                    counter.put()
                counter.counter += tallies[visitor]
                counter.put()
    return 'DONE (with %d task[s] logging %d visitor[s])\r\n' % (
        len(msgs), len(tallies))

```

主要應用程式處理常式 `root()` 沒有任何變更。HTML 範本檔案 `templates/index.html` 也不需要任何變更，因此這會包裝所有必要的更新。恭喜！您已使用 Cloud Pub/Sub 建立新的第 19 堂課應用程式！

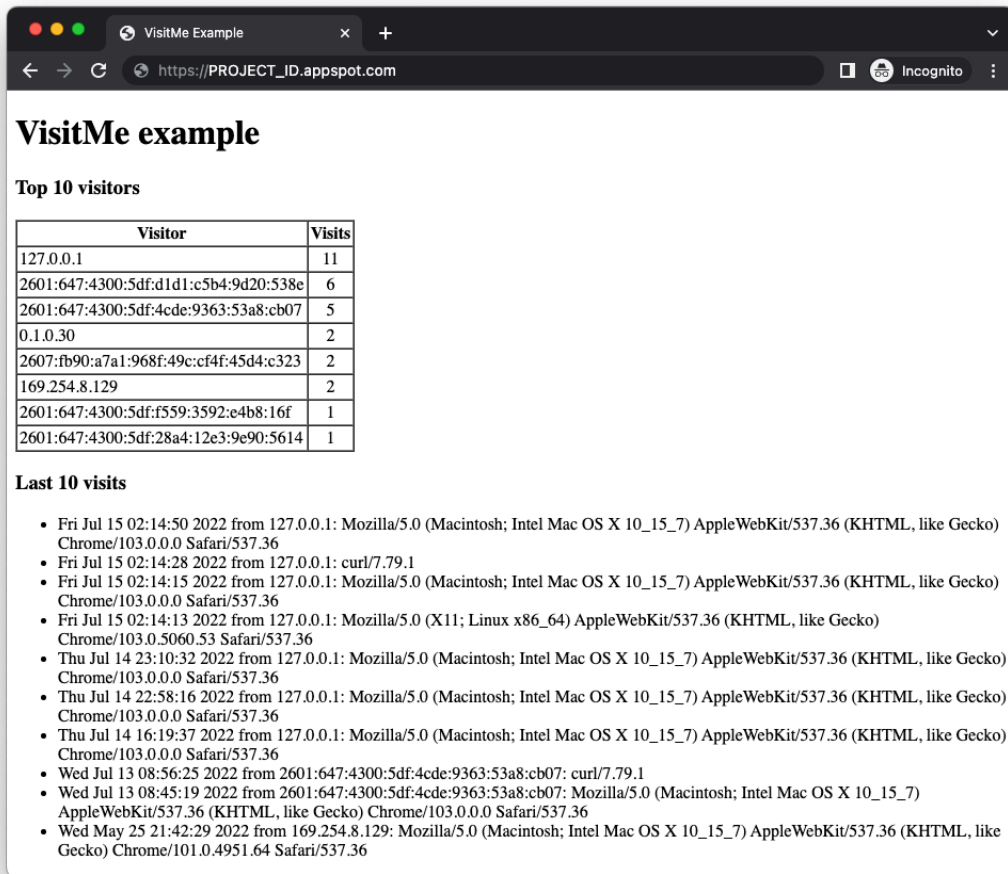
步驟 7 · 約 5 分鐘

7. 摘要/清除

部署應用程式，確認應用程式是否正常運作，以及是否會反映任何輸出內容。此外，請執行工作站來處理訪客人數。應用程式驗證完成後，請執行任何清理步驟，並考慮後續步驟。

部署及驗證應用程式

請確認您已建立 `pullq` 主題和 `worker` 訂閱項目。如果已完成上述步驟，且範例應用程式已準備就緒，請使用 `gcloud app deploy` 部署應用程式。輸出內容應與第 18 堂課的應用程式相同，但您已成功替換整個基礎佇列機制：



應用程式的網頁前端現在會驗證應用程式的這部分是否正常運作。雖然應用程式的這部分會成功查詢並顯示熱門訪客和最近造訪次數，但請注意，應用程式會註冊這項造訪，並建立提取工作，將這位訪客加入總計數。該工作目前已排入佇列，等待處理。

您可以透過 App Engine 後端服務、`cron` 工作、瀏覽 `/log`，或發出指令列 HTTP 要求來執行這項操作。以下是執行範例，以及使用 `curl` 呼叫工作站程式碼（請代入 `PROJECT_ID`）：

```
$ curl https://PROJECT_ID.appspot.com/log
DONE (with 1 task[s] logging 1 visitor[s])
```

更新後的計數會在下次造訪網站時反映。就是這麼簡單！

清除所用資源

一般

如果暫時不需要使用，建議[停用 App Engine 應用程式](#)，以免產生費用。不過，如果您想進一步測試或實驗，App Engine 平台提供[免付費配額](#)，只要不超出該用量層級，就不會產生費用。這是指運算費用，但您可能也需要支付相關 App Engine 服務的費用，因此請參閱[其定價頁面](#)瞭解詳情。如果這項遷移作業涉及其他雲端服務，則這些服務會另外計費。無論是哪種情況，請視需要參閱下方的「本程式碼研究室專用」一節。

為求完整揭露，部署至 App Engine 等 Google Cloud 無伺服器運算平台時，會產生[少量建構和儲存空間費用](#)。[Cloud Build](#) 和 [Cloud Storage](#) 都有各自的免費配額。儲存該圖片會耗用部分配額。不過，你所在的區域可能沒有這類免付費層級，因此請留意儲存空間用量，盡量減少潛在費用。您應審查的特定 Cloud Storage「資料夾」包括：

- `console.cloud.google.com/storage/browser/LOC.artifacts.PROJECT_ID.appspot.com/containers/images`
- `console.cloud.google.com/storage/browser/staging.PROJECT_ID.appspot.com`
- 上述儲存空間連結取決於您的 `PROJECT_ID` 和 `*LOC*`ation，例如，如果您的應用程式託管於美國，則為「us」。

另一方面，如果您不打算繼續使用這個應用程式或其他相關的遷移 Codelab，並想完全刪除所有內容，請[關閉專案](#)。

本程式碼研究室專用

下列服務是本程式碼研究室的專屬服務。詳情請參閱各項產品的說明文件：

- Cloud Pub/Sub 的不同元件都有免費方案，請先判斷整體用量，以便進一步瞭解費用影響，詳情請參閱[定價頁面](#)。
- [App Engine Datastore](#) 服務是由 [Cloud Datastore](#) (Cloud Firestore Datastore 模式) 提供，這項服務也有免費方案；詳情請參閱[定價頁面](#)。

後續步驟

除了本教學課程外，您還可以參考其他遷移單元，瞭解如何從舊版套裝組合服務遷移：

- [單元 2](#)：從 App Engine `ndb` 遷移至 Cloud NDB
- [單元 7 至 9](#)：從 App Engine 工作佇列 (推送工作) 遷移至 Cloud Tasks
- [第 12 至 13 個單元](#)：從 App Engine Memcache 遷移至 Cloud Memorystore
- [單元 15 至 16](#)：從 App Engine Blobstore 遷移至 Cloud Storage

App Engine 不再是 Google Cloud 中唯一的無伺服器平台。如果您有小型 App Engine 應用程式或功能有限的應用程式，並希望將其轉換為獨立的微服務，或是想將單體應用程式拆分成多個可重複使用的元件，這些都是考慮遷移至 [Cloud Functions](#) 的好理由。如果容器化已成為應用程式開發工作流程的一部分，特別是如果工作流程包含 CI/CD (持續整合/持續推送軟體更新或持續部署) 管道，建議遷移至 [Cloud Run](#)。下列單元會說明這些情況：

- 從 App Engine 遷移至 Cloud Functions：請參閱[第 11 堂課](#)
- 從 App Engine 遷移至 Cloud Run：請參閱[第 4 節](#)，瞭解如何使用 Docker 將應用程式容器化；或參閱[第 5 節](#)，瞭解如何在不使用容器、Docker 知識或 `Dockerfile`s 的情況下完成這項作業。

您可以選擇改用其他無伺服器平台，但建議先考量應用程式和用途的最佳選項，再進行任何變更。

無論您接下來要考慮哪個遷移模組，都可以在 [開放原始碼存放區](#) 存取所有 [Serverless Migration Station 內容](#) (程式碼研究室、影片、原始碼 [如有])。此外，該存放區的 `README` 也提供指引，說明要考慮哪些遷移作業，以及遷移模組的相關「順序」。

8. 其他資源

以下列出其他資源，供開發人員進一步瞭解這個或相關的遷移模組，以及相關產品。包括提供內容意見回饋的位置、程式碼連結，以及您可能會覺得實用的各種文件。

程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

下表提供第 18 堂 (開始) 和第 19 堂 (完成) 課程的存放區資料夾連結。

Codelab	Python 2	Python 3
Module 18	code	(不適用)
單元 19 (本程式碼研究室)	code	(與 Python 2 相同，但請使用 <code>app3.yaml</code> ，除非您已如上所述更新 <code>app.yaml</code>)

線上參考資料

以下是與本教學課程相關的資源：

App Engine Task Queue

- [App Engine 工作佇列總覽](#)
- [App Engine Task Queue 提取佇列總覽](#)
- [App Engine 工作佇列提取佇列完整範例應用程式](#)
- [建立 Task Queue 提取佇列](#)
- [Google I/O 2011 推出佇列影片 \(Votelator 範例應用程式\)](#)
- [queue.yaml 參考資料](#)
- [queue.yaml 與 Cloud Tasks 比較](#)
- [提取佇列至 Pub/Sub 遷移指南](#)

Cloud Pub/Sub

- [Cloud Pub/Sub 產品頁面](#)
- [使用 Pub/Sub 用戶端程式庫](#)
- [Pub/Sub Python 用戶端程式庫範例](#)
- [Pub/Sub Python 用戶端程式庫說明文件](#)
- [建立及管理 Pub/Sub 主題](#)
- [Pub/Sub 主題命名規範](#)
- [建立及管理 Pub/Sub 訂閱項目](#)

- [App Engine \(彈性\) 範例應用程式](#) (也可部署至標準環境；Python 3)
- [上述範例應用程式的存放區](#)
- [Pub/Sub 提取訂閱項目](#)
- [Pub/Sub 推送訂閱項目](#)
- [App Engine Pub/Sub 推送範例應用程式 \(Python 3\)](#)
- [App Engine Pub/Sub 推送範例應用程式存放區](#)
- [Pub/Sub 定價資訊](#)
- [Cloud Tasks 或 Cloud Pub/Sub ? \(推送與提取\)](#)

App Engine NDB 和 Cloud NDB (Datastore)

- [App Engine NDB 說明文件](#)
- [App Engine NDB 存放區](#)
- [Google Cloud NDB 說明文件](#)
- [Google Cloud NDB 存放區](#)
- [Cloud Datastore 定價資訊](#)

App Engine 平台

- [App Engine 說明文件](#)
- [Python 2 App Engine \(標準環境\) 執行階段](#)
- [在 Python 2 App Engine 上使用 App Engine 內建程式庫](#)
- [Python 3 App Engine \(標準環境\) 執行階段](#)
- [Python 2 和 3 App Engine \(標準環境\) 執行階段的差異](#)
- [Python 2 到 3 App Engine \(標準環境\) 遷移指南](#)
- [App Engine 定價和配額資訊](#)
- [推出第二代 App Engine 平台 \(2018 年\)](#)
- [比較第一代和第二代平台](#)
- [對舊版執行階段的長期支援](#)
- [說明文件遷移範例](#)
- [社群提供的遷移範例](#)

其他雲端資訊

- [在 Google Cloud Platform 上執行 Python](#)
- [Google Cloud Python 用戶端程式庫](#)
- [Google Cloud 「永久免費」 方案](#)
- [Google Cloud SDK \(`gcloud` 指令列工具\)](#)
- [所有 Google Cloud 說明文件](#)

影片

- [無伺服器遷移工作站](#)
- [無伺服器 Expeditions](#)
- [訂閱 Google Cloud Tech](#)

- 訂閱 [Google Developers](#)

授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。
